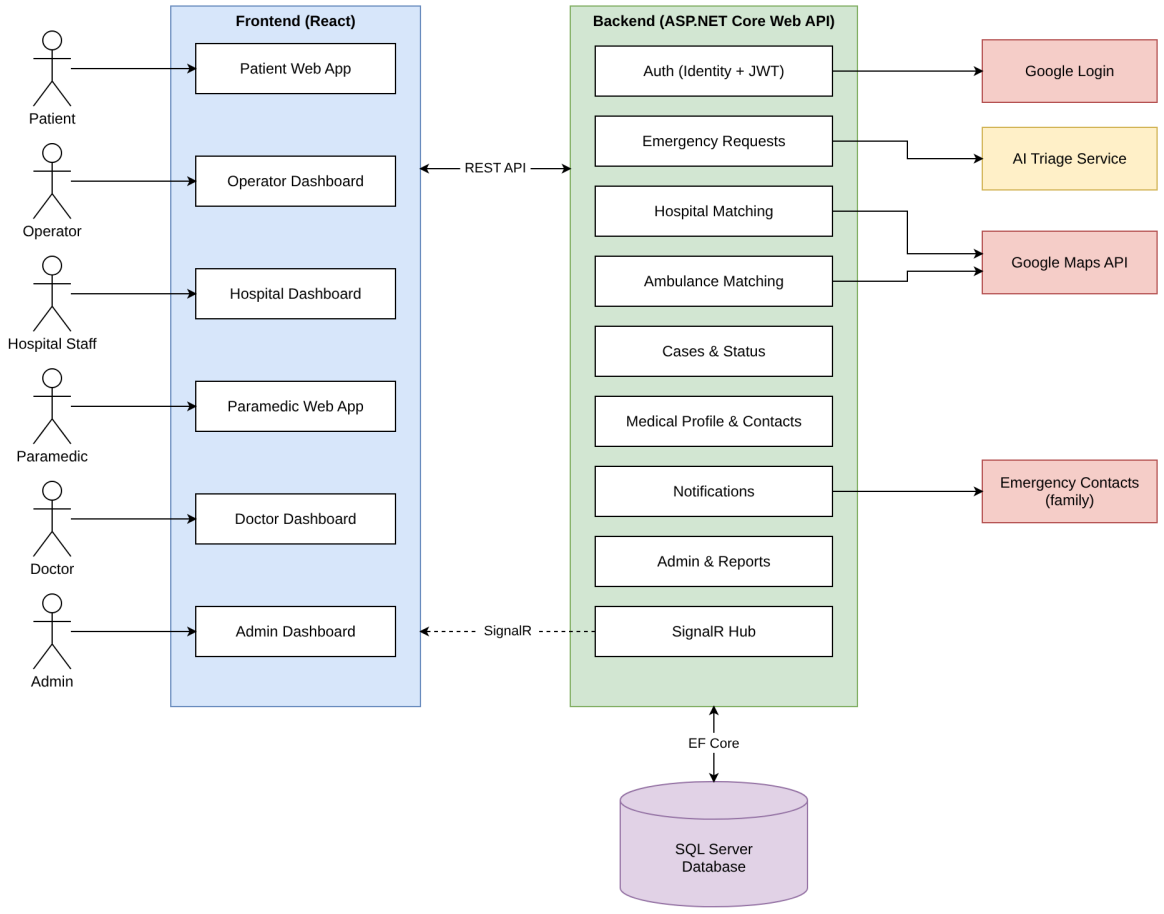
System Architecture

Smart Emergency Response & Healthcare Coordination Platform



1. Main Parts

- **Frontend (React):** the screens. Each role has its own pages.
- **Backend (ASP.NET Core Web API):** gets requests from the screens, does the logic and saves the data.
- **Database (SQL Server):** keeps all the data. The backend uses EF Core to talk to it.
- **Auth (Identity + JWT):** login, and each user sees only what his role allows.
- **SignalR:** sends updates to the screens right away without refresh.
- **AI Triage Service:** suggests the emergency type and priority. The operator decides.
- **Google Login and Google Maps:** sign in with Google, and the map and the distance. The GPS location itself comes from the browser on the patient's phone.

2. Frontend Screens

Role	Screen	What he does there
Patient	Patient Web App	Sends the emergency request, fills his medical profile and emergency contacts, tracks the ambulance
Operator	Operator Dashboard	Reviews requests, sends them to a hospital, follows all open cases
Hospital Staff	Hospital Dashboard	Accepts or rejects requests, sends an ambulance with a paramedic, updates free beds, closes the case
Paramedic	Paramedic Web App	Gets the job, sees the patient's info, updates the status
Doctor	Doctor Dashboard	Sees the case and the medical profile, writes notes
Admin	Admin Dashboard	Manages users, hospitals and ambulances, sees reports

All screens work on phone and desktop.

3. Backend Parts

Part	What it does
Auth (Identity + JWT)	Register, login, Google login and roles
Emergency Requests	Gets the patient's request with the GPS location (or the address he wrote if GPS is off) and asks the AI for a suggestion
Hospital Matching	Suggests the closest hospital with the right department and free beds. Used by the operator
Ambulance Matching	Suggests the hospital's own closest free ambulance. Used by hospital staff
Cases & Status	Keeps the case status and its history, and the doctor's notes
Medical Profile & Contacts	Allergies, diseases, medicines and the patient's emergency contacts
Notifications	Sends messages to the emergency contacts when the request is sent and when the case changes
Admin & Reports	Users, roles, hospitals, ambulances and reports
SignalR Hub	Pushes live updates to the screens

4. How the Parts Talk

- **Frontend to Backend:** REST API over HTTPS. Data is sent as JSON and every request has the JWT token.
- **Backend to Frontend:** SignalR, for live things like the ambulance on the map or "hospital accepted".
- **Backend to Database:** Entity Framework Core. The screens never talk to the database directly.
- **Backend to other services:** Google Login for sign in, Google Maps for the map and the distance, the AI service for triage.

5. Example: One Emergency Request

1. The patient presses the emergency button. The Patient Web App takes his GPS location from the browser (or he writes his address if GPS is off) and sends the request to the backend.
2. The backend saves it, sends a message to the emergency contacts, asks the AI service for a suggestion and shows the request on the Operator Dashboard by SignalR.
3. The operator confirms the type and priority.
4. Hospital Matching suggests the closest hospital and the operator sends the request to it.
5. Hospital staff accept it (or reject it, and the operator sends it to the next closest hospital).
6. Ambulance Matching suggests the hospital's closest free ambulance. Staff send it with a paramedic.
7. The paramedic gets the job and updates the status. The patient, the operator and the hospital see it live, and the emergency contacts get a message.
8. The hospital receives the patient, the doctor writes his notes and the hospital closes the case.